OPERATING SYSTEMS

Complete Course Notes with Practical Examples & Coloured Vector Flowcharts

Definitions + Flowcharts + Comparison Tables + Editor-style Code + Real Linux/Windows Commands | 5 Units

SYLLABUS COVERED

Unit 1 Introduction: definition, functions, evolution, types & desirable characteristics; OS services, system calls vs utility programs (with POSIX/Windows examples); layered structure	**Unit 2** File management: file concept/attributes, user vs system view, access methods; file-system modules; disk space allocation (contiguous/linked/indexed); directories & protection; disk scheduling (FCFS, SSTF, SCAN, C-SCAN, LOOK) with worked examples	
Unit 3 CPU scheduling: process concept & 5-state diagram, schedulers, algorithms (FCFS/ SJF/Priority/RR/Multilevel) & evaluation; threads & multiprocessor scheduling; memory management: partitioning, swapping, segmentation, paging, paged-segmentation, address translation, overlay & dynamic linking; virtual memory & demand paging, page replacement (FIFO/LRU/OPT), thrashing	**Unit 4** I/O systems: programmed/interrupt/DMA + double buffering; concurrency: critical section, semaphores, producer-consumer, readers-writers; deadlock: conditions, prevention, avoidance (Banker’s algorithm), detection & recovery	
Unit 5 Network, distributed & multiprocessor operating systems; case studies: UNIX/Linux & Windows NT; real-time & contemporary OS (containers: cgroups + namespaces, RTOS)	**Practical** Every unit includes real shell / syscall / Windows command examples and a worked numeric example	**How to use** Read in order; redraw each vector diagram; redo the numeric examples by hand

Self-study & exam-prep guide following the OS syllabus. All flowcharts are vector graphics - crisp and colourful on screen and in print.

TABLE OF CONTENTS

Introduction to Operating Systems definitio utility prog

File Systems, Directories & Disk Scheduling file con disk sched

CPU Scheduling, Threads & Memory Management process partitionin

I/O, Concurrency & Deadlock progra detection/

Distributed OS & Case Studies network RTOS & c

Study tip: redraw every coloured vector flowchart on paper, then re-derive the numeric examples (scheduling, paging, Banker) from memory.

UNIT 1 | INTRODUCTION TO OPERATING SYSTEMS

1.1 What is an Operating System?

OPERATING SYSTEM (OS)

An OS is a SYSTEM SOFTWARE that acts as an intermediary between the USER, APPLICATION programs and the HARDWARE. It manages resources (CPU, memory, disk, I/O), schedules processes, hides hardware complexity and provides a clean interface. Examples: Windows, Linux, macOS, Android, iOS.

os_check.sh

```
# practical: who sits where?
user -> applications (browser, games) -> OPERATING SYSTEM -> hardware (CPU/RAM/disk/NIC)
# on Linux, the OS you talk to most is the kernel:
uname -r        # kernel version       (e.g. 6.6.1-arch1-1)
whoami          # current user (OS tracks users & permissions)
free -h         # memory status: used / free / swap
top             # live running processes + CPU% (OS scheduler in action)
df -h           # mounted file systems (OS file management)
```

Layer	Who	Examples	Role
User	people	commands, GUI	requests a service
Application	programs	Chrome, WhatsApp	use OS services via APIs
OS services	kernel+daemons	scheduler, VFS, driver	allocate & protect resources
Hardware	devices	CPU, RAM, disk	raw machine resources

1.2 Functions, Evolution & Types

CORE FUNCTIONS (THE FIVE MANAGERS)

PROCESSOR management (schedule & dispatch), MEMORY management (allocate/free/protect), FILE management (create/read/write/secure files & directories), DEVICE management (control and schedule I/O), plus SECURITY & user management, and JOB/process accounting/statistics.

EVOLUTION - WHY EACH STAGE HAPPENED

1) 1940s-50s NO OS (manual, one job) -> 2) BATCH systems (monitor reads a queue of jobs, no user interaction, submit-card) -> 3) MULTIPROGRAMMING (several jobs in memory; if one waits for I/O another runs - CPU utilisation up) -> 4) TIME-SHARING (round-robin so everyone gets a slice; interactive) -> 5) MULTI-PROCESSOR, REAL-TIME (deterministic deadlines, robotics/aero), NETWORKED & DISTRIBUTED OS (resources across machines).

OS type	Key idea	Used for	Practical OS
Batch	jobs queued, no user at run	payroll, batch compute	early IBM 360 OS/360
Multitasking (preemptive)	CPU slices switched fast	desktop/laptop work	Windows, Linux, macOS
Real-time (RTOS)	guaranteed response time	airbags, robots, trading	FreeRTOS, VxWorks, QNX
Distributed	looks like ONE system spread over machines	datacentres, clusters	Kubernetes layer, Amoeba
Embedded	tiny footprint, single purpose	washing machine, car ECU	Linux-embedded, RTOS
Mobile	touch-driven, power aware, sandboxed apps	phones/tablets	Android, iOS

1.3 Desirable Characteristics & Features

DESIRABLE CHARACTERISTICS OF A GOOD OS

RELIABILITY (never crash silently), SECURITY (protect users/files from each other), EFFICIENCY (high throughput, low overhead), SCALABILITY (more cores/users/threads), PORTABILITY (runs on new hardware), MAINTAINABILITY (updates without restart), ACCOUNTING (usage statistics), RESPONSIVENESS & PERFORMANCE (low latency, high throughput).

OS LAYERED STRUCTURE (EXTENDED MACHINE CONCEPT)

```
                +---------------- APPLICATION & USER INTERFACE  ----------------+
                |  shell / GUI       (where users type commands)
  ---- SYSTEM CALL PROGRAMS ----------------------------------------
                |  OS SERVICES - process, file, memory, device, security
  ---- KERNEL (core: scheduling, VFS, drivers, IPC)  ----  MICROKERNEL?
                +---------------- H A R D W A R E -----------------+
   layering: each level calls the level below through well-defined interfaces
```

1.4 OS Services - types & how they are provided

TYPES OF SERVICES

Program EXECUTION (load & run, terminate), file & I/O (open/read/write), COMMUNICATION (IPC between processes), ERROR DETECTION (hardware/software/spec failures), RESOURCE ALLOCATION, ACCOUNTING, and PROTECTION & SECURITY. These are grouped as: user-interface, program execution, I/O operations, file-system manipulation, communications, error detection, allocation, accounting, protection.

Service	User sees it as...	Kernel does...	Example syscall
Program execution	run my app	create process, load, dispatch	fork(), execve()
I/O	file saves / print	device driver + buffer	read(), write()
Filesystem	folders & files	VFS + disk driver + locking	open(), lseek()
Communication	apps talk	sockets/shared memory	socket(), mmap()
Error detection	no crash, clean msg	trap handling, logs	signal(), sigaction()
Protection	my files private	permissions, isolation	chmod(), setuid()

HOW SERVICES ARE PROVIDED: SYSTEM CALLS VS UTILITY PROGRAMS

SYSTEM CALLS = the official kernel API (privileged, fast; e.g., read(), fork()). They are the ONLY way a user program touches the hardware - enforced by CPU privilege modes (user mode vs kernel mode via trap instructions). UTILITY PROGRAMS = ordinary programs built ON TOP of system calls (e.g., mkdir, ls, cp, top). Layering: app -> utility program -> system call -> kernel -> hardware.

SysCall.java.txt

```
// system call == controlled entry to kernel. In C on POSIX:
#include <unistd.h>
if (fork() == 0) { // syscall #57 : create a child process
    execlp("ls", "ls", NULL); // syscall group: replace this code with 'ls'
}
// In Java you do not call syscalls directly - the JVM does it:
new File("data.txt").exists(); // -> JVM -> openat() syscall -> kernel
```

syscalls_practical.txt

```
Linux counts system calls (e.g. on x86-64): ~350+ calls.
Open the manual:   man syscalls
Count them at run time:   strace -c ls   # lists read/write/openat/brk etc used by 'ls'
Windows equivalent: Win32 API (CreateProcess, ReadFile) -> NT system calls.
```

Exercise: *1) What is the difference between a system call and a function call? 2) Name 4 OS services you saw while using a phone today. 3) Draw the layered structure of your own OS with 3 utilities and 3 syscalls. 4) Batch vs time-sharing - which gave better CPU utilisation and why?*

UNIT 2 | FILE SYSTEMS, DIRECTORIES & DISK SCHEDULING

2.1 File Concept & the two views

FILE

A logical unit of related information stored on secondary storage - the OS presents bytes, records or tree-structured data and hides where/what they physically are. Attributes: name, type, location, size, owner, protection, timestamps. A file has STRUCTURE (sequence of words/records) but the OS often treats it as an uninterpreted byte stream (UNIX).

View	Who uses it	What a file looks like
USER view	programmer / user	logical data - bytes, records, or tree of fields; API: open/read/write/close
SYSTEM view	kernel + drivers	blocks on disk; Map: logical blocks <-> physical sectors via inodes/FAT
File operations	everyone	create/delete/open/read/write/seek/truncate; licensed by access methods

USER VS SYSTEM VIEW OF A FILE (LINUX INODE VS DIRECTORY ENTRY)

```
user: open("a.txt", READ) ---> VFS (common interface) ---> FAT / inode lookup
     file block 0..n  --->  logical->physical mapping  ---> disk driver
ls -l a.txt   # size 5120 bytes octal permission -rw-r--r-- owner group
Linux keeps file META in inode: perms, times, block list; name lives in directory
```

DISK VS TAPE ORGANIZATION - WHY RANDOM ACCESS CHANGES DESIGN

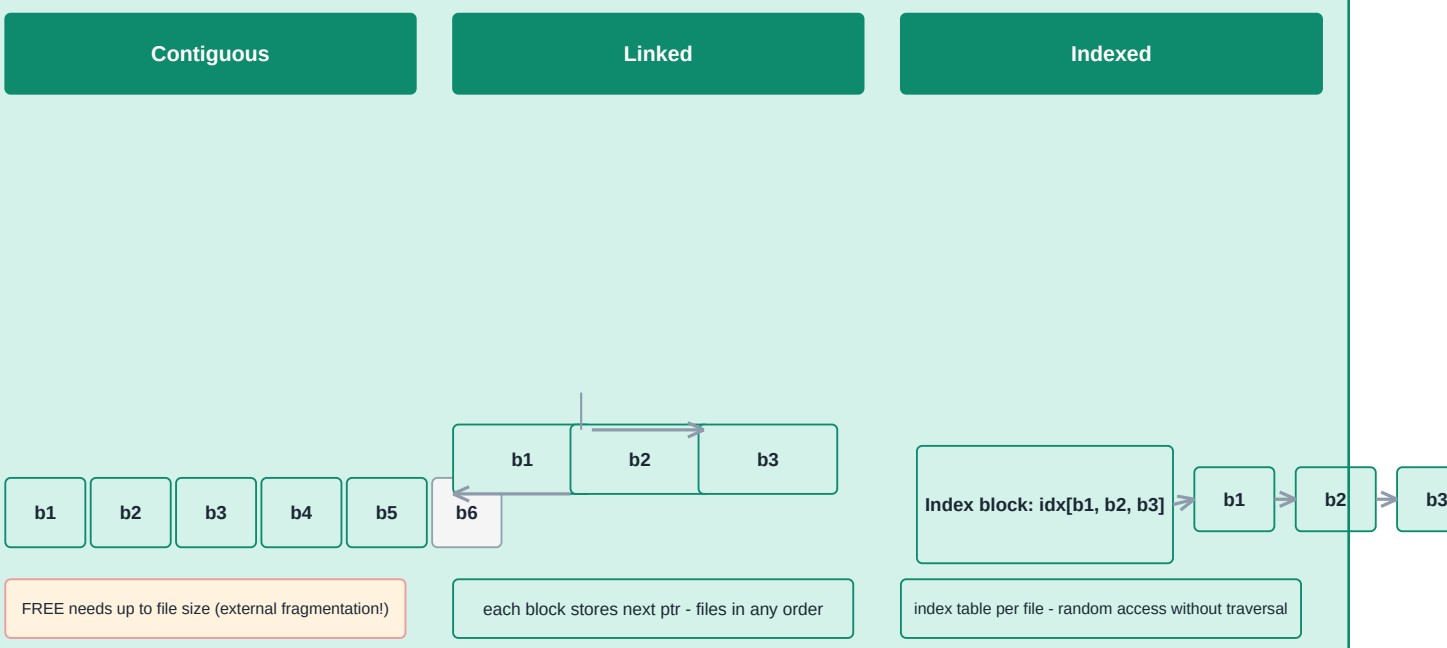
```
TAPE: sequential only - records read in order; rewinds to re-read.
DISK (random): blocks addressable - seek + read.
Access methods: sequential (read next), direct/random (block n), indexed (by key).
```

2.2 Modules of a File System & Disk Space Allocation

FILE-SYSTEM LAYERS (FROM VFS DOWN)

Application -> LOGICAL file system (metdata, directory) -> FILE-ORGANISATION module (maps logical blocks) -> BASIC I/O (buffers, block driver) -> DEVICE (disk) driver. Each layer hides details and gives a clean API to the one above.

Disk Space Allocation: Contiguous vs Linked vs Indexed



Contiguous = fast sequential + random, but no growth; Linked = no FRAG but slow random; Index risk = index block loss

three allocation schemes + their trade-offs (always asked)

Scheme	Fast access	Fragmentation	Growth	Used by
Contiguous	yes (seek-less)	external	hard	CD/DVD ISO, old FAT
Linked	no (must walk)	none	easy	FAT-12/16/32
Indexed (inode)	yes via table	small (index), no external	good	ext4, NTFS, ReFS

2.3 Directory Structures & File Protection

DIRECTORIES

A directory is a TABLE mapping names -> file descriptors (inode/FAT entry). 1-level (all in one) is simplest; TWO-level (per-user + system) adds isolation; TREE (hierarchical paths) is universal; ACYCLIC GRAPH allows shared dirs (links); GENERAL GRAPH allows cycles but needs garbage collection. Teachers: name uniqueness, path resolution, directory caching.

Protection mechanism	Idea	Practical
Access control list (ACL)	per (user, group, others) rights	ntfs perms, chmod-like lists
Permission bits	rwx for own/group/world	chmod 755 file.sh
Capabilities	holder shows "ticket"	tokens, sudo -u
Encryption at rest	bytes useless without key	BitLocker, LUKS, filevault

file_syscalls.sh

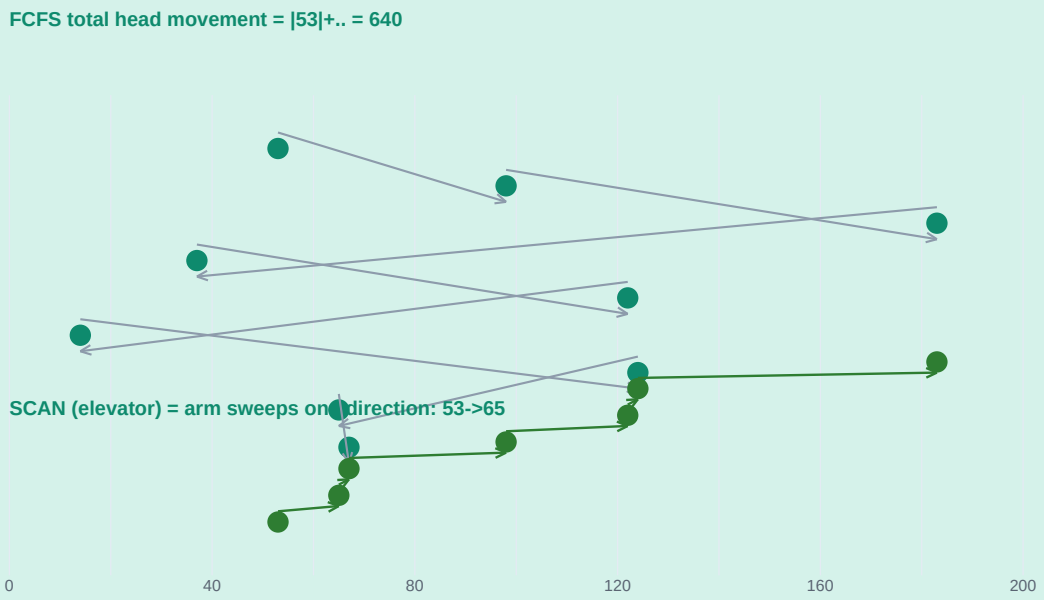
```
# file management system calls you should know:
cat > x.txt <<'EOF'           # create via shell
hi
EOF
touch x.txt                   # create/update timestamp
ls -li x.txt                  # inode number, perms
stat x.txt                    # meta view (size, blocks, atimes...)
chmod 750 x.txt               # rwx for owner, r-x group, --- others
mv x.txt dir/                 # rename/move = directory update
rm x.txt                      # delete (unlink)
find / -name '*.pdf'          # traverse directory tree
binary syscall: open(), close(), read(), write(), lseek(), ftruncate(), unlink(), chmod()
```

2.4 Disk Scheduling Algorithms

WHY SCHEDULE THE DISK HEAD

Disk latency = SEEK time (move head) + ROTATIONAL delay + TRANSFER time. Seek is mechanical (milliseconds) so reorder the pending request QUEUE to cut total head movement - just like CPU scheduling. Algorithms trade TOTAL movement vs FAIRNESS/starvation.

Disk Scheduling: FCFS vs SCAN (example)



then reverses and serves the rest; total ~236 in this sweep (incl. reversal)

seek = head travel; algorithms trade total movement vs starvation

Algorithm	Rule	Total movement*	Problem
FCFS	serve in arrival order	640 (example)	no starvation, no optimisation
SSTF (shortest seek first)	closest cylinder next	~236	starves far requests
SCAN (elevator)	sweep one way, then back	~236 per sweep	wait at edges (~208/2)
C-SCAN	sweep one way, wrap head	even wait	wraps wasted seek back

Algorithm	Rule	Total movement*	Problem
LOOK / C-LOOK	reverse only at LAST pending req	least waste	most used in practice

PRACTICE NUMBERS

Given head at 53, requests [98, 183, 37, 122, 14, 124, 65, 67]: FCFS = |98-53|+|183-98|+... = 640 cylinders; SSTF = 53->65->67->37->14->98->122->124->183 = 236. Always draw the number line first - it is the whole trick.

Exercise: *1) requests {55, 43, 22, 95, 17} head 50: find FCFS, SSTF, SCAN (+ to 200). 2) Why can a file become fragmented and what does defrag do? 3) For an inode-based fs explain 1 file read path end-to-end.*

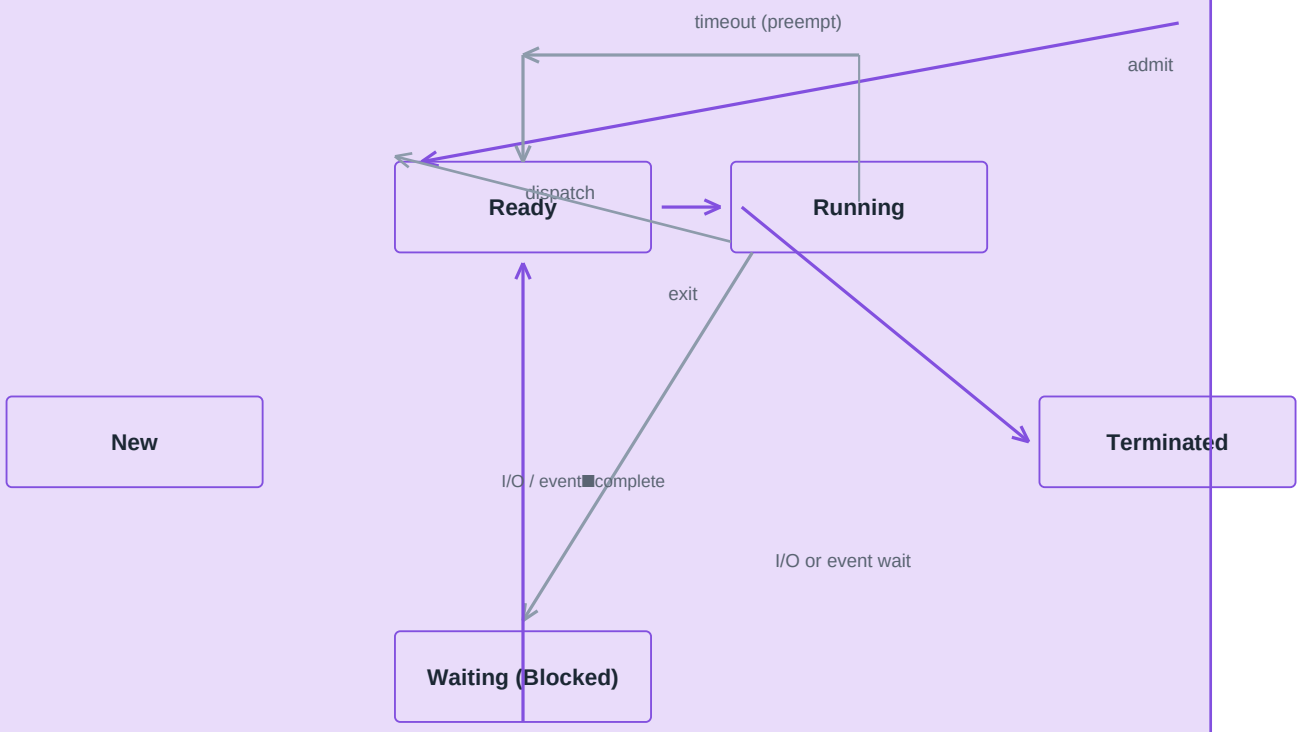
UNIT 3 | CPU SCHEDULING, THREADS & MEMORY MANAGEMENT

3.1 Process Concept & States

PROCESS

A program in EXECUTION (only ONE process ownership of CPU at an instant on a single-core). Process state = current activity: NEW -> READY -> RUNNING -> WAITING/Terminated, with all the transitions. Kept in the PROCESS CONTROL BLOCK (PCB): PID, program counter, register save area, scheduling info, memory limits, open-file table.

Process 5-State Model

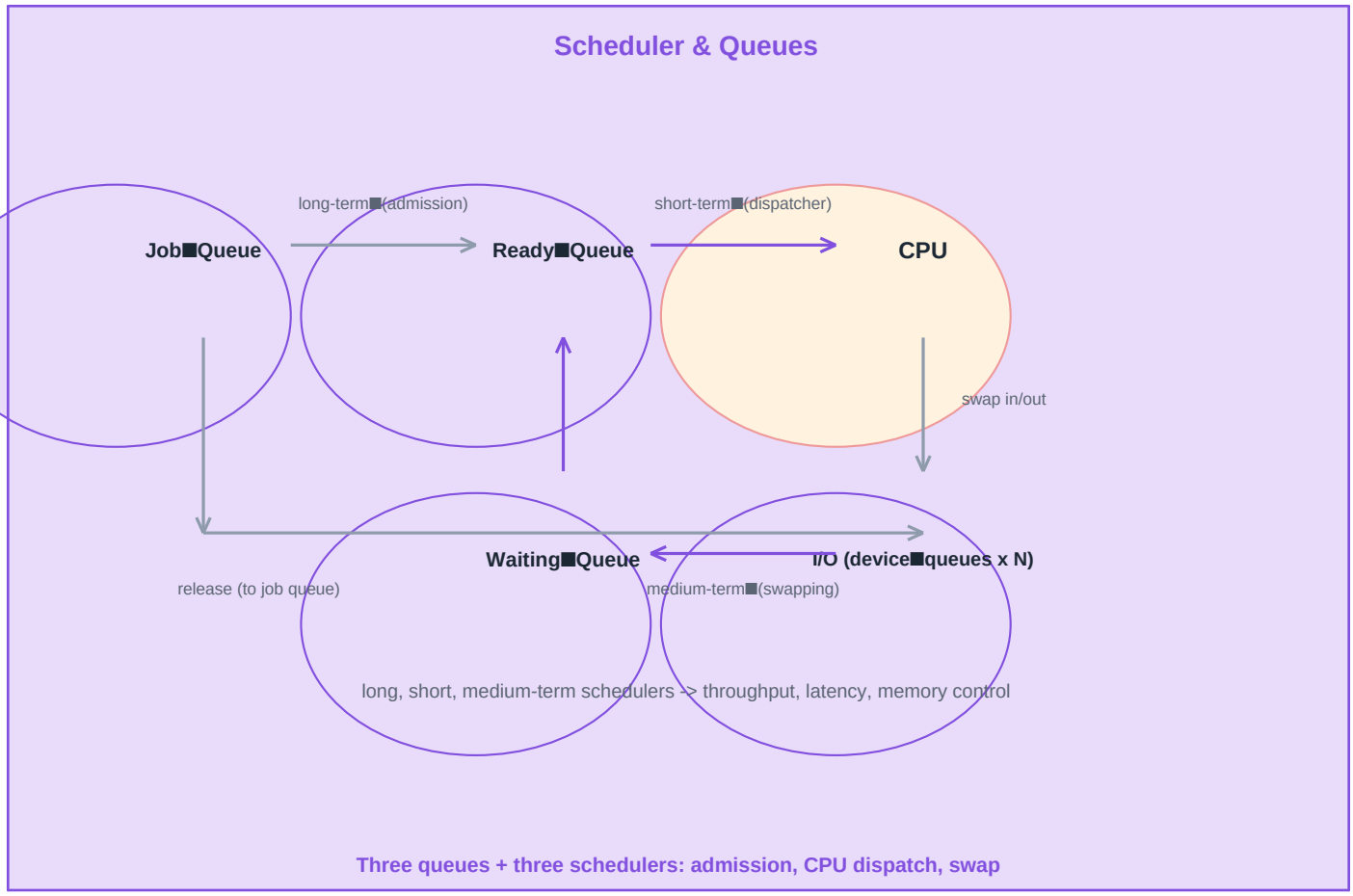


process = program in execution: PCB = PID, state, PC, registers, memory, open files

OS 5-state process transition lamp (standard exam question)

SCHEDULERS

LONG-TERM (job) scheduler selects WHICH jobs to ADMIT into the mix (low frequency); SHORT-TERM (CPU/dispatcher) picks the next READY process to RUN every time slice (very high frequency); MEDIUM-TERM does SWAPPING in/out to balance memory.



3.2 CPU Scheduling Algorithms

CONCEPTS & METRICS

Arrival/burst times; CPU-bound vs I/O-bound. Lap measures: TURNAROUND = completion - arrival; WAITING = turnaround - burst; RESPONSE = first CPU - arrival; THROUGHPUT = jobs/unit time; Preemptive (stop running process) vs non-preemptive.

FCFS & SJF (EXAMPLE WITH NUMBERS)

FCFS: join queue; avg waiting can be huge (convoy effect). SJF: shortest burst first - optimal average waiting NON-preemptive; SFJF (SRTN) preemptive version. Example bursts P1=6, P2=8, P3=7, P4=3: SJF order P4,P1,P3,P2 -> waiting 0,3,9,16 avg 7 vs FCFS 0,6,14,21 avg 10.25. Starvation: long jobs may wait forever.

SJF VS ROUND-ROBIN QUICK EXAMPLE

```
Gantt (SJF):  | P4 |   P1   |    P3    |      P2      |
times:         0    3       9          16             24
waiting: P4=0, P1=3, P3=9, P2=16   avg = 7.0   (best possible)
Gantt (RR q=4): |P1|P2|P3|P4|P1|P3|P2| -> fairness, response goodness
```

Algorithm	Type	Starve?	Best for
FCFS	non-preemptive	no	batch, simplicity
SJF / SFJF	non/ pre	yes (long)	min avg waiting (proof: exchange arg)
Priority	both	yes (low)	real-time, interactive priority
Round Robin (q=...)	preemptive	no	time-sharing; q->0 like CPU, q->inf like FCFS
Multilevel Queue / Feedback	preemptive	maybe	mixed interactive + batch

EVALUATION OF ALGORITHMS

ANALYTICAL (mathematical: average waiting for FCFS known), SIMULATION (event-driven with real traces), IMPLEMENTATION (practical but costly) and typical measurement: CPU utilisation = busy/(busy+idle), throughput, turnaround time distribution. "Prediction" for SJF uses exponential averaging of past bursts.

3.3 Multiple-Processor Scheduling & Threads

MULTIPROCESSOR & THREAD CONCEPTS

MULTIPROCESSOR: symmetric (SMP - one run queue per CPU or shared) vs asymmetric (master/slave), processor affinity (keep a process on its CPU for cache warmth), load balancing. THREAD = lightweight process - one process, MANY threads sharing code/data but with own stack+PC. USER threads (fast, managed in user space; can hide blocking) vs KERNEL threads (kernel schedules them; heavier) - modern OS map 1:1 (Linux NPTL); Java Thread maps to a kernel thread via the JVM.

Threads.java

```
// Java threads (kernel-backed in practice):
class Bank implements Runnable {
    public void run() { for (int i = 0; i < 5; i++) System.out.println(Thread.currentThread().getName()); }
}
Thread t = new Thread(new Bank(), "t1");
t.start(); // schedules a kernel thread
t.join(); // wait for it (like wait() in POSIX)
// Linux (pthreads -> kernel threads): ps -eLf shows LWP column = kernel threads
```

3.4 Memory Management Techniques

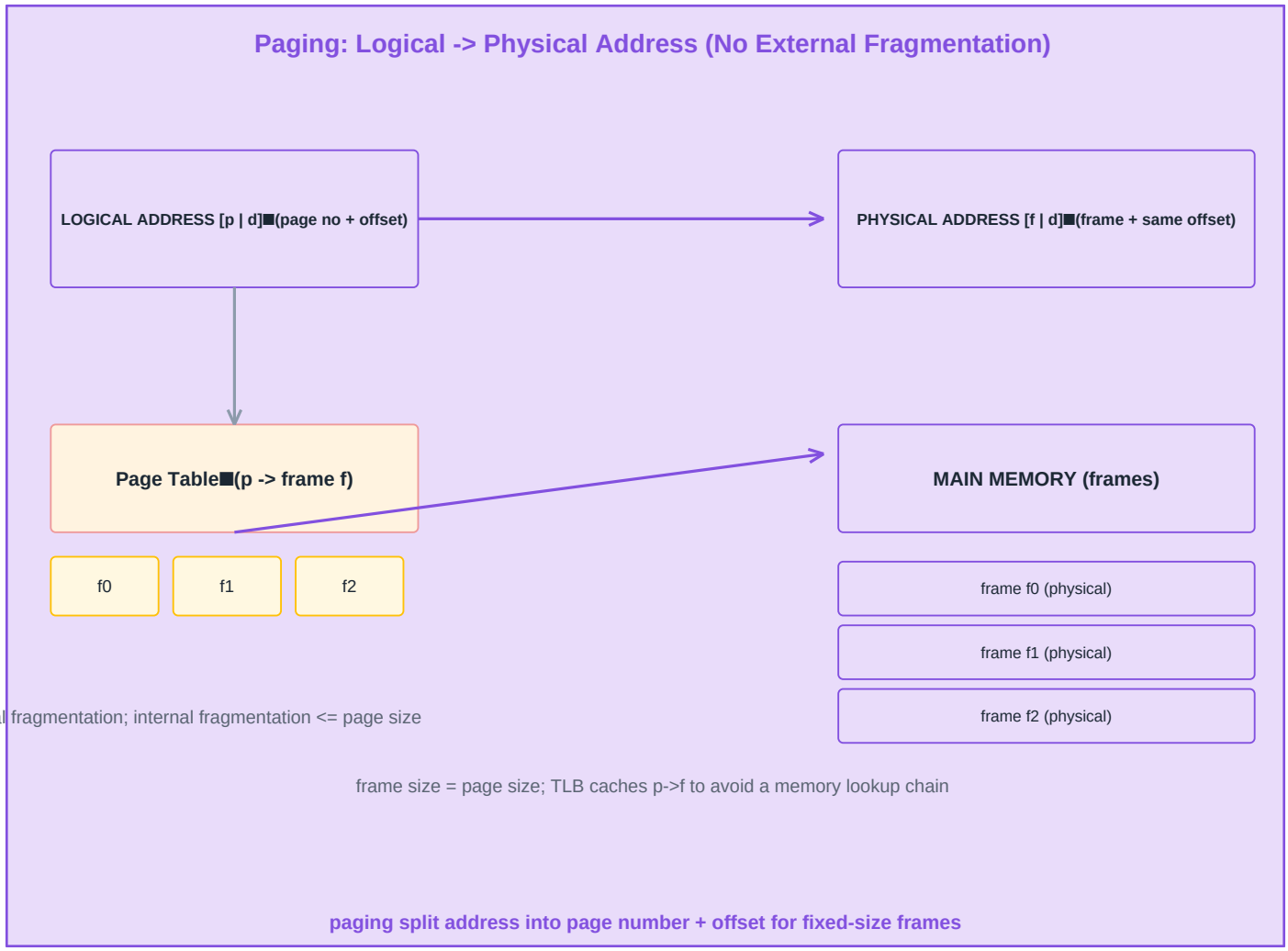
CONTIGUOUS & PARTITIONING

PARTITIONING: fixed (equal/unequal; internal fragmentation) or dynamic (variable; external fragmentation needs compaction). PLACEMENT policy: first/best/worst fit. SWAPPING: move whole process to disk when idle to free RAM for others.

Technique	External frag.	Internal frag.	Address translation	Comment
Fixed partition	no	yes	base only	simple, wasteful
Dynamic partition	yes	no	base+limit	needs compaction
Segmentation	yes	no	base+limit per segment	user view of modules
Paging	no	<= page size	page table [p|d]	hardware TLB
Paged segmentation	no (pages)	small	segment table + page table	PDP-11/Intel

ADDRESS TRANSLATION NAILED

In paging: logical address [p | d] -> page number p -> page-table lookup -> frame f -> physical f*pageSize + d. TLB caches recent p->f mappings to make it fast (one memory access instead of two). Page size enabled (4 KB typical; huge pages 2 MB for large apps) is a design trade-off.



3.5 Large Programs: Overlay, Dynamic Linking & Virtual Memory

RUNNING PROGRAMS LARGER THAN RAM

OVERLAY: programmer partitions code into segments swapped as needed (manual, obsolete). DYNAMIC LINKING: shared libraries loaded at run time, one copy in memory shared (DLLs, .so - saves memory, updates easy). DYNAMIC LOADING: routines loaded only when called. VIRTUAL MEMORY solves "not enough RAM": give each process its own huge virtual address space (e.g. 48-bit), bring pages in ON DEMAND.

DEMAND PAGING & PAGE REPLACEMENT

On a page fault: valid bit=0 -> trap, fetch from swap, update table, restart. REPLACEMENT when no free frame: FIFO (Belady anomaly exists), LRU (best approximation - use reference bit / second-chance), OPT (theoretical). Working set = set of pages in use; thrashing = working set size > RAM.

mem_practical.txt

```
# practical: watch your own virtual memory
ps -o pid,rss,vsz,cmd -p $$          # RSS = resident (in RAM), VSZ = virtual size
top -b -n1 | head -n 14              # MEM% per process, swap use
cat /proc/meminfo                    # MemTotal, SwapTotal, Dirty pages
./bigalloc.sh &                       # demand paging: only touched pages fault in
free -h                               # buffers/cache vs used - OS reclaims
```

EXAM TIP

exponential prediction & LRU are the two favourite numerical exam topics - practise them.

Exercise: *1) bursts {3,6,1,7} order P1..P4 - FCFS vs SJF vs RR(q=3) turnarounds. 2) Draw the 5-state diagram and label 7 transitions. 3) Convert logical address 4098 (page size 1 KB) with page table [0->2,1->5,..] to physical. 4) Why does LRU beat FIFO in practice (give a reference-string example)?*

UNIT 4 | I/O, CONCURRENCY & DEADLOCK

4.1 Principles of I/O Hardware & Software

I/O HARDWARE REALITY

Devices are asynchronous and orders of magnitude slower than the CPU (disk ~1-10 ms vs CPU nanoseconds). An I/O request needs: DEVICE CONTROLLER (chip per device), registers (control, status, data) mapped either in the I/O port space or MEMORY-MAPPED, and INTERRUPTS so the CPU is not blocked polling. The OS provides a uniform DEVICE-DRIVER interface: block vs character vs network devices.

THREE TRANSFER MODES (CONCEPT + TRADE-OFF)

PROGRAMMED I/O: CPU polls status then reads data into memory byte-by-byte - simple but wastes CPU. INTERRUPT-DRIVEN: device interrupts when ready; CPU still copies data (1 word per interrupt) - better but interrupts flood the CPU at high data rates. DMA (Direct Memory Access): controller copies a whole BLOCK straight to memory, ONE interrupt at the end - essential for disks, NICs (Gbps).

Mode	Who copies	Interrupts	CPU overhead	Used by
Programmed (polling)	CPU	none	100% busy while I/O	legacy cheap chips
Interrupt-driven	CPU per word	per word/byte	high at high speed	slow consoles
DMA	controller	once per block	low (also double-buffered)	SSDs, NICs, disks

DMA DATA PATH + DOUBLE BUFFERING

```
CPU  <--- init (addr,count) --->  DMA controller  <--- n/w --->  device
EOS signal:  DMA -> interrupt -> CPU:  "block 512 bytes done"
double buffering: while DMA fills B1, CPU works on B2 -> no device-idle gaps
practical:  iostat -x 1   # see await, svctime, %util per disk
```

4.2 Concurrency, Critical Section & Synchronisation

CONCURRENT PROCESSES

Two processes are concurrent if their execution OVERLAPS in time; on a single CPU this means their INSTRS are interleaved in some arbitrary order -> RACE on shared variables. Need MUTUAL EXCLUSION inside the CRITICAL SECTION (region touching the shared variable). Four requirements: mutual exclusion, progress (no one outside blocks entry), bounded waiting, and no busy-wait ideal with semaphores.

Race.java

```
# classic race - the "lost update"
int counter = 0;
// shared by 2 threads: counter++ is really READ / ADD / WRITE (3 steps)
T1: READ counter(0) ADD 1(1) WRITE(1)
T2: READ counter(0) ADD 1(1) WRITE(1)
result = 1, expected 2 !!
// fix with a semaphore / mutex protecting the critical section, or an atomic
synchronized (lock) { counter++; } // Java: one CS at a time
```

SEMAPHORES

S is a non-negative integer with two atomic ops: WAIT(P): if S>0 S-- else BLOCK; SIGNAL(V): S++ and wake one waiter. Counting semaphore = resource pool (S printers); BINARY (mutex) = locked/unlocked. They give minimal busy waiting and are the base of every synchronisation pattern (here classically: readers-writers, dining-philosophers, producer-consumer).

SEMAPHORE OPS + BOUNDED-BUFFER PRODUCER-CONSUMER

```
P(S): [ if S==0 -> BLOCK ]  else S--        V(S): S++, WAKEUP one process
Producer:  produce();  P(empty); P(mutex);  add;  V(mutex); V(full);
Consumer:  P(full);   P(mutex);  take;      V(mutex); V(empty);  consume();
deadlock-free because lock order (empty then mutex) never nests in reverse
```

4.3 Deadlock: Necessary Conditions, Prevention/Avoidance/Recovery

DEADLOCK

A set of processes each holding a resource and waiting for one held by another - none can progress. Four NECESSARY conditions: 1) MUTUAL EXCLUSION, 2) HOLD AND WAIT, 3) NO PREEMPTION, 4) CIRCULAR WAIT. Safe state (banker's algorithm) = there EXISTS an order in which everyone finishes; allocation that would leave the system UNSAFE is rejected.

Strategy	What you do	Cost / problem	Example
Prevention	break one of the 4 conditions (e.g. order all resources, request all at once)	low utilisation	resource ordering rules
Avoidance	banker's algorithm: only allocate if SAFE	needs future max need info	banker textbook solution
Detection	wait-for graph cycle? kill & restart one of them	finding + restart cost	debuggers, DBMS
Recovery	preempt (rollback) / kill victim / starve killer	lose work	kill -9 watchdog

BANKER'S ALGORITHM MINI-EXAMPLE

Processes P0,P1,P2; resources A,B (totals 10,5) currently allocated (2,2); available = (8,3). Max wants: P0(3,2) P1(6,1) P2(8,3). Need = Max-Allocated: P0(1,0) P1(4,1) P2(6,3). Is P2's request (3,3) SAFE? Available would be (5,0) - P0 needs (1,0) ok finish -> release -> (8,3); P1 needs (4,1) ok -> finish -> (10,4); P2 finishes last; YES safe order P0,P1,P2 -> the request is granted. First, do this on paper; the safe CHECK is a DFS over "can someone finish".

conc_practical.sh

```
# practical concurrency + deadlock in real shells / OS
ps -eLf | wc -l                # huge thread count = kernel threads alive
lsof -p $$ | head              # open files (resources held by a process)
kill -0 PID                    # probe: does the process still exist?
flock /tmp/lock bash -c 'sleep 3; echo done'   # file lock = semaphore on file
# Windows:  tasklist /V ;  netstat -ano | findstr 8080  (ports = resources)
```

CONCEPT CHECK

Deadlock vs Starvation: deadlock = circular waiting that never ends; starvation = a process waits forever while others progress (e.g. SJF long-job, low priority).

Exercise: *1) Write a producer-consumer with three semaphores and prove it deadlock-free. 2) Why is deadlock IMPOSSIBLE if every process acquires resources in ONE fixed order? 3) Banker: available (3,3,2), Max/Allocation given - find a safe sequence. 4) Distinguish programmed, interrupt, DMA with 2 points each.*

UNIT 5 | DISTRIBUTED OS & CASE STUDIES

5.1 Network / Distributed & Multiprocessor Operating Systems

NETWORK OS VS DISTRIBUTED OS

NETWORK OS (NOS): each machine runs its own OS; users NOTICE machines, access remote files by NFS/SMB; no global scheduling; simple, but no transparency. DISTRIBUTED OS: the set of machines looks like ONE computer - PROCESS MIGRATION, GLOBAL scheduling, TRANSPARENT remote file system, IPC as messages/RPC across nodes, and DEADLOCK/STABILITY detection across nodes. Multiprocessor OS extends symmetric vs asymmetric schemes to scheduling + memory sharing (shared-memory multiprocessor with cache-coherence).

System	Kernels#	User sees	Design complexity	Real world
Single-OS	one node	one machine	low	your laptop
Network OS	many, same/different	a LAN of machines	medium (protocols only)	Windows/Linux LAN
Distributed OS	one logical over many	ONE machine	high (migration, replication)	research; GFS piece
Multiprocessor OS	one (shared memory)	one fast machine	needs locking + affinity	server SMP boxes

THE DISTRIBUTED-OS ILLUSION: ONE SYSTEM ACROSS MANY CPUS

```
Distributed = same OS kernel image on every node + MESSAGE passing (RPC) across nodes
    node1  <----RPC---->  node2 : flow = call -> stub -> marshall -> socket -> unmarshall
Transparency levels: location (path hides node) >> replication (copies hidden) >> failure
Distributed-deadlock: wait-for graph spans nodes - prevented by time-stamped locks.
```

5.2 Case Study: UNIX / LINUX

Aspect	UNIX/Linux answer
Structure	kernel (scheduler, VFS, TCP/IP, drivers) + system call table + user procs; monolithic with loadable modules (LKM)
Processes	fork() -> exec() -> exit(); scheduler = O(1)/CFS with nice values
Memory	paging + demand paging, virtual address 47-bit; no swap-in/out per PROCESS, per-page
File	inode tree; /dev, /proc, /sys virtual filesystems; chmod/octal permissions, ACLs
IPC	pipes, signals (kill), sockets, SysV message queues, shared memory via mmap
I/O	everything is a (possibly virtual) file -> uniform driver interface
Boot chain	BIOS/UEFI -> GRUB -> kernel (initramfs) -> systemd -> login shell

fork.c

```c
// the classic fork/exec/exit single process in C (POSIX) - one question, every exam:
#include <unistd.h>
int main() {
    pid_t p = fork(); // 0 in child, pid in parent
    if (p == 0) execlp("ls","ls",NULL); // child becomes 'ls'
    else wait(NULL); // parent waits child's exit
}
// shell sees: ps -ef | grep ls then echo $? -> exit status
```

5.3 Case Study: MS Windows (NT family)

Aspect	Windows answer
Structure	hybrid kernel (kernel + HAL + executive) + Win32 API; drivers via WDM/UMDF

Aspect	Windows answer
Processes	CreateProcess, threads (fibers), priority 0-31, job objects, UAC integrity
Memory	VirtualAlloc 64-bit; pages, working set (trim by priority), AWE
File	NTFS: MFT, ACL-based security, journaling (log file)
IPC	named pipes, mail slots, shared memory (file mapping), COM
I/O	IRP (I/O Request Packets) in layers: filter -> FSD -> disk -> hardware
Boot chain	UEFI/BIOS -> bootmgr -> winload -> ntoskrnl -> services (svchost)

os_id.txt

```
# spot the OS in action (cross-platform check)
uname -a / ver / Get-ComputerInfo          # which OS, build
systeminfo | findstr /i "os version"       # Windows detailed
tasklist | findstr chrome                   # processes running
netstat -ano | findstr :443                 # sockets = IPC + network
powershell Get-Service spooler              # services = daemons
# both OSes support: preemptive multitask, virtual memory, ACL security.
```

5.4 Contemporary & Real-Time OS

MODERN OS ADDITIONS

CGROUPS (container CPU/memory limits - the basis of Docker), NAMESPACES (isolated views of processes/files/net - containers), kexec/overlayfs for fast boot, live patching, scheduler EAS (energy-aware), ASLR + mitigations for security, RTOS notions: deadline scheduling (EDF), priority inversion solved with priority-inheritance mutex (e.g. FreeRTOS, QNX, even in Linux PREEMPT_RT).

modern_os.txt

```
# containers use OS primitives, not a new OS:
docker run -it ubuntu bash        # = namespaces + cgroups + chroot/overlay
unshare -p -f bash                # new PID namespace (mini-container, no docker)
ulimit -s 8192                    # per-process resource limit (OS enforced)
cgroup:  /sys/fs/cgroup/.../cpu.max   # 2 cpus max for that group
```

SUMMARY LINE

Compare UNIX kernel vs NT executive vs microkernel (Minix/QNX): monolithic=fast/coupled, micro=clean/reliable but IPC cost, hybrid=pragmatic.

Exercise: *1) Tabular-distinguish NOS vs distributed OS (6 points). 2) Trace Linux boot from power-on to your shell. 3) Explain how a container gets its own process tree using namespaces. 4) RTOS: what makes a deadline scheduler different from CFS?*